Python Imperativo

- Operadores
- Tipos de dados
- Atribuição
- E/S
- Comando condicional
- Comando de repetição
- Função

Operadores de expressões

- Multiplicativos: *, /, %
- Aditivos: +, -
- Relacionais: <, <=, >, >=
- Igualdade: ==, !=
- Lógicos: and, or, not

Comentários

- Os comentários são inseridos no código-fonte para fornecer explicações aos humanos
- Não têm nenhum efeito nas funcionalidades da classe

```
# quantidade de creditos cursados
    creditos = 134
```

Comentários

- Comentários mais detalhados são escritos em múltiplas linhas
- Começam com três caracteres “””
- E terminam com o par “””
- “”” esse comentário é um mais longo, devido .... “””

Tipos de dados

Primitivos:

- **boolean**
- **int, long,**
- **float**
- **complex**

de referência:

- **objetos, listas, tuplas**

Comando de atribuição

- Em linguagem Python

  variável = expressão

  Ou

  variável: tipo = expressão

- Exemplos
  - **altura = 1.90**
  - **filhos: int = 3**
  - **netos = filhos * 2 + 3**

Variáveis e suas declarações

- Cada linguagem fornece ao programador uma relação de tipos padronizados para suas variáveis.
- A linguagem Python fornece tipos primitivos para variáveis inteiras, reais e lógicas
- Permite que o programador crie novos tipos

Variáveis e suas declarações

- Declaração de várias variáveis em Python:
  - **variável, variável, ..., variável = valor, valor,...valor**
- Exemplos
  - x1: int = 10 ou x1 = 10
  - y1: float = 5.4 ou y1 = 5.4
  - v1, v2, v3 = 1, 5, 23
  - f1, f2 = 2.45, 3.14
- Notem que em Python não é necessário declarar o tipo da variável.
- x1 = 10 também é válido

Variáveis inteiras

Tipo	Nº de bytes	Intervalo de valores
int	2 (palavra)	[-32768, +32767]
short	2	[-32768, +32767]
long	4	[-2147483648, +2147483647]
unsigned	2 (palavra)	[0, +65535]
unsigned short	2	[0, +65535]
unsigned long	4	[0, +4294967295]

Variáveis reais

Tipo	precisão	Intervalo de variação do expoente
float	7 dígitos decimais	[-38, +38]
double	16 dígitos decimais	[-308, +308]

Variáveis caracteres

Tipo	Intervalo dos valores
char	[128, +127]
unsigned char	[0, 255]

Variáveis lógicas

- A linguagem Python disponibiliza um tipo de variável específico para armazenar valores lógicos
- Os valores são: True e False
- Os operadores lógicos em Python são: and, or, not
- Nome do tipo de dados lógico: bool

Saída de dados em Python

- Função print, que na verdade não imprime nada em papel.
- Exibe um valor na tela.
- print("Hello, World!")

- s: float = 45.7
- print(“a area vale:”, s)

Entrada de dados em Python

- Existe uma função interna em Python para obter entrada do usuário: input
- nome = input("Entre com seu nome: “)
- Ou
- nome: str = input("Entre com seu nome: “)
- Em Python, sempre ler cadeia de caracteres na entrada do teclado, então precisamos converter para valores numéricos
- n1: int = int(input(“Entre com um valor inteiro:”))
- n2: float = float(input(“Entre com um valor real:”))

Fazendo escolhas

condição booleana a ser testada — fornece um resultado T*rue* (verdadeiro) ou F*alse* (falso)

palavra-chave‘if’

ações se a condição for verdadeira

```
if(realiza algum teste) :
    Segue as instruções aqui se o teste forneceu um resultado verdadeiro

else :
    Segue as instruções aqui se o teste forneceu um resultado falso
```

palavra-chave ‘else’

ações se a condição for falsa

Comando if-else

```
if (condição) :
    # a ser executada se condição é TRUE
    Comando c1
    Comando c2
    ....
else :
    # a ser executada se condição é FALSE
    Comando d1
    Comando d2
    ....
```

Exemplo 1

```
if x % 2 == 0:
   print (“x eh par”)
else:
   print (“x eh impar”)
```

Exemplo 2

```
if x % 2 == 0:
   return True
else:
   return False
```

Concatenação de string

- + é o operador de concatenação de string em Python
- Exemplos:
- ‘POO’ + ‘com Python’ -> ‘POOcom Python’
- ‘Resposta: ‘ + str(42) -> ‘Resposta: 42’
- return “0” + str(valor)
- return “” + str(valor)

Iteração

- Freqüentemente, queremos realizar algumas ações em um número arbitrário de vezes.
  - Por exemplo, imprimir todas as notas na agenda. Quantas existem?
- A maioria das linguagens de programação inclui instruções de *loop* para tornar isso possível.
- Python tem duas categorias de instruções de loop (while e for).
  - Vamos começar com o loop while.

O loop `while`

- Sintaxe semelhante à linguagem C
- Utilizado freqüentemente para iterar por um número arbitrário de vezes.
- Utilizado para iterar por uma lista.

Pseudocódigo do loop `while`

Forma geral de um loop while

Palavra-chave `while`

Teste booleano

```
while(condição do loop):
    corpo do loop
```

Instruções a serem repetidas

Exemplo de pseudocódigo para imprimir todas os itens

```
while(há pelo menos mais um item a ser impresso):
    mostre o próximo item
```

O loop `while`

```
# Soma de 0 a 99
soma=0
contador=1
while contador < 100:
    soma=soma+contador
    contador=contador + 1
print (soma)
```

Iterando com for

- O loop **for** trabalha com list da seguinte maneira:

for VARIABLE **in** LIST:

  BODY

Iterando com for

- exemplo:

```
friends = ["Joe", "Zoe", "Adam", "Josy", "Zuki", “Gray"]
for friend in friends:
    print(friend)
```

Iterando com for

- Sintaxe:

```
for <referência> in <sequência>:
    <bloco de código>
else:
    <bloco de código>
```

Iterando com for

```
# Soma de 0 a 99
soma=0
for contador in range(1, 100):
    soma=soma + contador
print (soma)
```

Iterando com for

- A função range(m, n, p), é muito útil em laços,
- pois retorna uma lista de inteiros, começando em m e menores que n, em passos de comprimento p,
- que podem ser usados como sequência para o laço.

Definição de funções

- Forma geral:

```
def nome_da_função (lista de parâmetros...):
    corpo da função
    return [expressão]
```

Exemplo 1

```
def escrevame ( str ):
    print (str)
    return
```

Chamada da função

```
# agora vc pode chamar a função escrevame
escrevame("primeira chamada para...")
escrevame("segunda chamada para…")
```

Essas chamadas ficam no programa principal

Programa principal - prog1.py

```
# isto é um comentário
def escrevame( str ):
    print (str)
    return
# agora vc pode chamar a função escrevame no programa principal
escrevame("primeira chamada!")
escrevame("segunda chamada!")
escrevame("Hello World!")
```

Hora-Trabalho_Aula_Extra

Faça um programa principal em Python que receba a temperatura em graus Celsius e apresente-a em graus Fahrenheit, de acordo com a fórmula F = 1.8*C + 32

Fazer uma função em Python para verificar se um número é maior que 200 ou menor do que 100

Faça uma função em Python que calcule a soma dos números ímpares entre 1 e um limite superior definido pelo usuário.

Faça um programa principal para testar cada função

OBS: essa hora-trabalho não deve ser enviada para o professor